MOI

MOI BUILDERS · SESSION 8

Access policies

Fine-grained control over your own account.

Friday 4 September · 4pm IST · live on devnet

START HERE

You give an agent a job. Where does the record live?

It books, buys, researches, reports. Every one of those leaves a trace: what it spent, what it did, how many times. That trace is about *you*.

So the first question is not how to restrict the agent. It is where the record of its work is kept.

HOW IT WORKS TODAY

The app keeps its own copy of you.

On Ethereum a contract owns its storage. Off chain, the same shape: the service owns the row. Either way the record about you lives inside the thing you were using.

SCATTERED

One copy per service

Ten agents, ten separate records of what you did. No single view.

NOT YOURS

You are a row

You cannot read it all, move it, or take it with you.

NOT YOURS TO DELETE

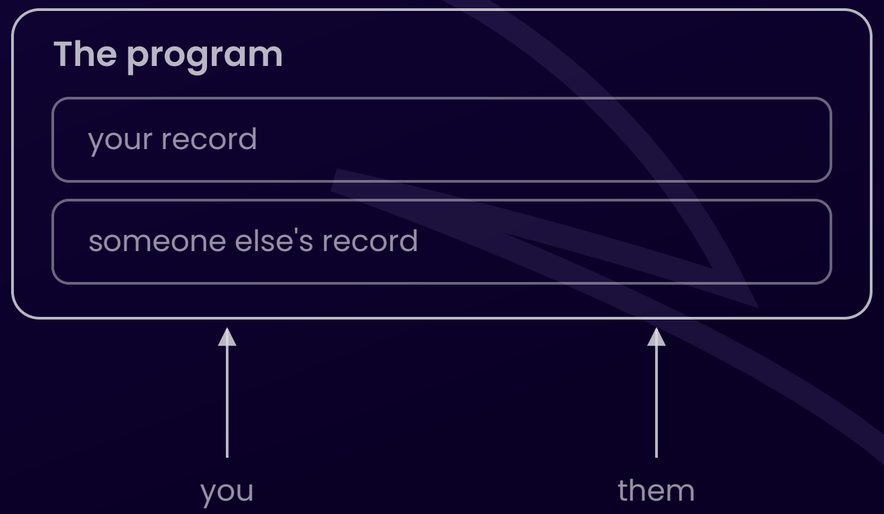
Their database, their rules

Removing the record means asking them to do it.

So MOI moved the data.

Your record lives on your account.

ETHEREUM



MOI

WHY THAT IS WORTH DOING

One account. Every agent you ever hire reads and writes there.

ONE PLACE, AND IT IS YOURS

Not ten copies of you

Every agent's record of what it did for you sits on one account you control, instead of scattered across ten services that each own their slice.

AGENTS COMPOSE

The next one picks up where the last left off

A new agent reads the history the previous one wrote, because they read the same account. Nobody has to build an integration between them.

SWAP WITHOUT LOSING ANYTHING

Fire the agent, keep the record

The history was never the agent's to take. Replace it tomorrow and the new one starts with everything the old one knew.

THE PART NOBODY ASKS ABOUT

If your data is on your account, programs must be able to write to accounts they do not own.

On Ethereum the question never comes up. Your account holds a balance and a nonce. It has no storage a contract can write into. Data lives inside the contract.

So MOI is not relaxing a safety rule. It is creating a capability that did not exist.

Open that capability and you immediately owe an answer: who is allowed to, and for what?

WHY NOT JUST WRITE A RULE

The agent holds its own key.

Put the rule in a program and the agent has to choose to call it. Any limit it looks up is one it can decline to look up. You are not constraining it. You are asking it nicely.

THE ANSWER

One record, on your account.

The protocol reads it before any foreign write lands. The program asking never touches it.

WHICH PROGRAM

The logic allowed to write

Named by its logic id, not a slot and not a person.

WHAT IT MAY DO

Mutate your storage

The one action live on the network today.

WHO IS BEHIND IT

The origin of the call

Pin the grant to one agent. Nobody else qualifies.

THE LOGIC

Thirty lines. No permission checks.

```
coco Ticker

state actor:              // each participant's counter lives on THEIR account
    counter U64

endpoint dynamic TickMy() -> (counter U64):
    mutate c <- Ticker.Sender.counter:
        c += 1

endpoint dynamic TickAny(participant Identifier) -> (counter U64):
    mutate c <- Ticker.Actor(participant).counter:   // a FOREIGN write
        c += 1

// abridged — full 30 lines in the repo
```

No `if sender == owner`. No owner field. No guard. All of that lives on the account.

THE ATTEMPT

What the agent actually sends.

```
// signed by the AGENT's own key — nobody can stop this part
const ticker = await getLogicDriver(TICKER_ID, agent.wallet);

await ticker.routines.TickAny(OWNER_ID).send({
  // declares what it will touch — a heads-up, not a permission
  participants: [{ id: OWNER_ID, lock_type: LockType.MUTATE_LOCK }],
  fuel_limit: 1500,
});
```

A real one, sent with no policy in place: `0x9fc1eff1…c9d4cbe1` — accepted, mined, charged 86 fuel.

THE GRANT

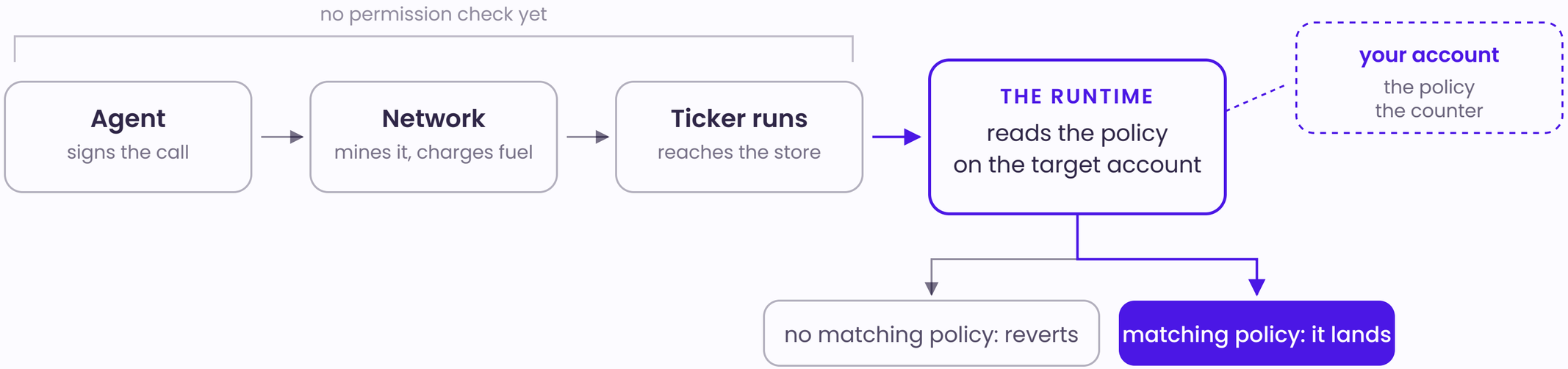
One interaction, signed by the owner.

```
await new Access(owner.wallet)
  .storage(TICKER_LOGIC_ID)              // WHICH program may write
  .allow(AccessAction.STORAGE_MUTATE)    // WHAT it may do
  .origin(access.callers(AGENT_ID))      // WHO must be behind it
  .create()
  .send();
```

The agent is not asked. It cannot decline. It finds out when it tries.

WHERE THE CHECK HAPPENS

No permission check until the write itself.



The interaction is signed, mined and charged either way. The runtime is the thing that refuses, so there is no version of the agent that skips this step.

LIVE DEMO

Same call. Same key. Three times.

1 · NO POLICY

Refused

counter unchanged

2 · POLICY GRANTED

Allowed

counter +1

3 · POLICY REVOKED

Refused

counter unchanged

WHAT THE CHAIN SAYS

```
// receipt status: 1 — mined, charged, reverted

builtin.AccessError:
actor is not allowed to write
into other actor's storage
```

Not a script deciding to stop. The runtime refused the write at the instruction, and the interaction is public, on the ledger, and paid for.

REVOCATION

Grant in one interaction. Revoke in one.

No redeploy. No key rotation. No asking the agent to behave. The next write simply fails.

HOW OTHERS ENFORCE

Everyone stops arbitrary writes. The ladder is where the rule lives.

	ENFORCED BY	UNIT OF PROTECTION
Ethereum	The contract's own code	Whatever that code says
Solana	The protocol	Per program
MOI	The protocol	Per account — set by you

HONEST SCOPE

Storage today.

STORAGE · LIVE

ASSET · RESERVED

LOGIC · RESERVED

KEY · RESERVED

"This agent may spend up to 500" is the shape of the thing, already in the type system. The mechanism you saw is the part that carries over.

Run it yourself.

github.com/sarvalabs-adithya/MOI-Webinars · session-8-policies

One funded devnet wallet · voyage.moi.technology

MOI Builders · the participant layer for AI agents